

Notes

Functions :

A function is a **block of statements that performs specific tasks** and is used to reduce **redundancy** in code

- Avoid code repetition
- Improve readability
- Easy debugging & maintenance

Definition Syntax

Functions are defined using `def`, optionally take parameters, and can return a value

```
def func_name (parameter1,parameter2):    #Define function
    # code block
    return Value
```

```
func_name(argument1,argument2) #Calling the function
```

```
def sum (a,b):
    return a+b
```

```
Total = sum(7,8)
print(Total)
```

```
15
```

Types of functions in Python →

Built-in Functions

Already available in Python

Examples: `print()` , `len()` , `type()` , `range()`

User-Defined Functions

Created by users using `def`

1 No parameter, no return

```
defhello():  
    print("Hello")  
  
hello()
```

2 No parameter, with return

```
defget_number():  
    return 10  
  
print(get_number())
```

3 With parameter, no return

```
defgreet(name):  
    print("Hello", name)  
  
greet("Lewis")
```

4 With parameter, with return

```
defadd(a, b):  
    return a + b  
  
print(add(2,3))
```

Lambda Function →

This is an **anonymous function in Python with no name**. It consists of a single expression and can be used as arguments.

```
def sum (a,b):  
    return a+b
```

```
Total = sum(7,8)  
print(Total)
```

OR

```
sum = lambda x,y: (x+y)
```

```
double = lambda x: (x*2)  
cube = lambda x: (x*x*x)  
print(sum)  
print(double)  
print(cube)
```

#Lambda as Argument

```
def appl(fx,value):  
    return 6 + fx(value)
```

```
print(appl(lambda x:x*x, 7))    #fx will get lambda as arg and val is 7 so 6 +  
7*7 = 55
```

Default Parameters →

Default values are used **when argument is not passed**.

Default parameters must be **at the end**.

- *Acceptable:* `sum( a,b=8 )` or `sum( a=7,b=8 )` .
- *Not Acceptable:* `sum( a=7,b )`

```
def sum (a=7,b=8):
    return a+b
```

```
Total = sum(2,3)
print(Total)
Total = sum()
print(Total)
```

```
5
15
```

Ex ⇒

#Print length of list

```
def length(numbers):
    ListLength = len(numbers)
    return ListLength
```

#Print elements of list

```
def ListElement(numbers):
    for i in numbers:
        print(i)
        i += 1
```

#Print factorial of number

```
def fact(a):
    value = 1
    while a>0:
        value = value*a
        a = a-1
    return value
```

```
List = [1,2,3,4,5,6,7,8,9]
```

```
print(length(List))
ListElement(List)
print(fact(5))
```

```
9
1
2
3
4
5
6
7
8
9
120
```

Recursion →

A function calling itself.

Must have a **base condition** to stop execution.

```
def show(n):
    if n == 0:      # Base case to terminate the flow
        return
    print(n)
    show(n-1)
```

```
show(5)
```

```
5
4
3
2
1
```

When a recursive function is called:

- Each function call is **stored in the call stack**
- New calls keep stacking **until the base condition is met**

- Once the base condition is reached, **no new calls are made**
- The function then **returns back one by one**, removing each call from the stack
- Any code written **after the recursive call** runs while returning

Think of recursion like **entering rooms one by one**.

```
Go to room5 → print5  
Go to room4 → print4  
Go to room3 → print3  
Go to room2 → print2  
Go to room1 → base condition reached → print"END"
```

Now you start **coming back**:

```
Comeout of room1  
Comeout of room2  
Comeout of room3  
Comeout of room4  
Comeout of room5
```

Each time you come out, Python **removes that function call from the stack**.

```
def sum (a):  
    if a ==0:  
        return 0  
    return a+sum(a-1)  
  
print(sum(7))
```

File I/O (input / output) :

Python enables operations like reading and writing data to files.

File Types

1. **Text files:** Files like `.txt` , `.docx` , or `.log` .
2. **Binary files:** Files like `.mp4` , `.mov` , `.png` , or `.jpeg`

Open , read and close file →

We must open a file before reading or writing to it.

- Syntax: `f = open ( "file_name" , "mode" ) .`
- Files must be closed using `f.close()` after operations
- Python opens text files by default (`t` mode).

file_name = sample.txt or path of file with file name

mode = read (`r`) , write (`w`)

```
f = open("Hello.txt", "r")  # Open the file

data = f.read()             # Reading the file
data = f.read(7)           # Read 1st 7 characters
line1 = f.readline()       # Read line by line
print(data)
print(type(data))
f.close()                  # Closing the file

Hello World!.
We are programming in Python
<class 'str'>
```

The `mode` determines the type of operation and how the file content is handled.

Mode	Description	Behavior
r	Read	Opens file for reading; file must already exist . Pointer starts at the beginning.

w	Write	Opens file for writing; creates new file or overwrites existing content.
a	Append	Opens file to append data; keeps existing content and adds new data to the end. Pointer starts at the end.
r+	Read and Write	Opens file for reading and writing; does not erase content. Writing starts from the beginning, removing existing characters.
w+	Read and Write	Opens file for reading and writing; clears old content first.
a+	Read and Append	Opens file for reading and appending.
Binary Modes	<code>rb</code> , <code>wb</code> , <code>ab</code>	Same as <code>r</code> , <code>w</code> , <code>a</code> but used for binary data (images, videos).

Writing file →

```
f = open("New.txt", "w")
f.write("Hello there")    # Creates new file and add this data
f.close()

f = open("New.txt", "a")
f.write("\nWe are learning Python")
f.close()
```

Pointer at start in read mode

Pointer at last in append mode

with syntax →

The `with open()` syntax is preferred because **it automatically closes the file**, removing the need for an explicit `f.close()` call

```
with open("OnePiece.txt", "r") as f:
    data = f.read()
    print(data)
```

Deleting file →

Files can be deleted using the `os` module

```
import os  
  
os.remove("New.txt")
```

Error Handling →

An **error** occurs when Python cannot run your code properly, often leading to the program stopping (crashing)

Example:

```
print(10 / 0)
```

Output:

```
ZeroDivisionError: division by zero
```

Here, Python **stops the program** because dividing by 0 is not allowed.

Why do we need error handling?

Without error handling:

- Program **crashes**
- User sees scary error messages

With error handling:

- Program **does not crash**
 - You can show a **friendly message**
 - Program continues running
-

try and except (Basic Error Handling)

Simple structure

```
try:
    # code that may cause an error
except:
    # code that runs if error happens
```

Example

```
try:
    x = 10 / 0
except:
    print("Something went wrong")
```

Output:

```
Something went wrong
```

Python **tried** to run the code.

When error happened, it **jumped to except**

Catching specific errors

Instead of catching all errors, catch **specific ones**.

Example: ZeroDivisionError

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("You cannot divide by zero")
```

Example: ValueError

```
try:
    num = int("abc")
```

```
except ValueError:  
    print("Please enter a valid number")
```

Multiple except blocks

You can handle **different errors differently**.

```
try:  
    a =int(input("Enter a number: "))  
    result =10 / a  
    print(result)  
except ValueError:  
    print("That is not a number")  
except ZeroDivisionError:  
    print("Number cannot be zero")
```

else block (Runs if NO error)

The `else` block runs **only if try succeeds**.

```
try:  
    a =int(input("Enter a number: "))  
    result =10 / a  
except ZeroDivisionError:  
    print("Cannot divide by zero")  
else:  
    print("Result is:", result)
```

finally block (Always runs)

The `finally` block **always executes**, error or no error.

Example

```
try:
    file = open("test.txt","r")
    print(file.read())
except FileNotFoundError:
    print("File not found")
finally:
    print("Closing program")
```

Used for:

- Closing files
- Closing database connections
- Cleanup tasks

Common Python Errors

Error Type	When it happens
<code>ZeroDivisionError</code>	Divide by zero
<code>ValueError</code>	Wrong value type
<code>TypeError</code>	Wrong data type operation
<code>FileNotFoundError</code>	File does not exist
<code>IndexError</code>	List index out of range
<code>KeyError</code>	Dictionary key missing

Example with real-life logic

Without error handling

```
age =int(input("Enter age: "))
print("Your age is", age)
```

If user enters `abc` → Program crashes.

With error handling

```
try:
    age =int(input("Enter age: "))
    print("Your age is", age)
except ValueError:
    print("Age must be a number")
```

Raising your own error (raise)

You can create **your own error**.

```
age =int(input("Enter age: "))

if age <18:
    raise ValueError("Age must be 18 or above")

print("You are eligible")
```

Object Oriented Programming →

OOP models **real-world entities** using classes & objects.

Class →

A class is like a blueprint that contains information and features

(e.g., `Tata cars` is a class)

Object (Instance) →

An object is a specific instance of a class

(e.g., `Nexon` or `Punch` are objects of the `Tata cars` class)

```

class Student:          # Class
    name = "Raj"
    age = 20
    Standard = 12

s1 = Student()          # Object / Instance
print(s1)
print(s1.name)
print(s1.age)
print(s1.Standard)

<__main__.Student object at 0x000001B3D6FDB1A0>
Raj
20
12

```

Constructor → [`__init__`]

A **constructor** is a special method called `__init__`.

It runs **automatically when an object of a class is created**.

- Almost every class uses an `__init__` method to **initialize (set up) object data**
- The `self` parameter refers to **the current object being created**
- `self` is always the **first parameter** in the constructor
- We usually use a **parameterized constructor** to pass values while creating an object
- If you don't define a constructor, Python provides a **default one automatically**
- If a constructor with parameters is defined, it **overrides the default constructor**

```

class Student:
    name = "Raj"
    age = 20

```

```

Standard = 12
def __init__(self):          #Constructor
    print("This is constructor")
    print(self)

s1 = Student()

This is constructor
<__main__.Student object at 0x0000026E0948B0B0>    #Self i.e. s1

```

```

class Car:
    def __init__(self, name, model, year):
        self.name = name          #It will create var name in c1
        self.model = model
        self.year = year

c1 = Car("BMW", "X5", 2020)
print(c1.name)
print(c1.model)
print(c1.year)

c2 = Car("Audi", "A6", 2021)
print(c2.name)
print(c2.model)
print(c2.year)

BMW
X5
2020
Audi
A6
2021

```

Class & Instance attributes →

How we access them →

- **Class attribute** → `ClassName.attribute`

Example: `Car.brand`

- **Instance (object) attribute** → `objectName.attribute`

Example: `c1.model`

What's the difference?

- **Class attributes**
 - Shared by **all objects** of the class
 - Same value for every object
 - Defined **outside the constructor**
 - Helps save memory because only **one copy** exists
- **Instance attributes**
 - Specific to **each object**
 - Values can be different for every object
 - Defined using `self` inside `__init__`

If an **object attribute** and **class attribute** have the same name, **object (instance) attribute** gets higher priority.

```
class Car:

    brand = "BMW" # Class attribute (same for all cars)

    def __init__(self, model, year):
        self.model = model # Instance attribute
        self.year = year   # Instance attribute

c1 = Car("X5", 2020)
print(Car.brand) # Accessing class attribute
print(c1.model)
print(c1.year)

c2 = Car("X3", 2021)
print(c2.model)
print(c2.year)
```

BMW

X5

2020

X3

2021

Methods →

Methods are functions that **belong to a class** and work on the **data of an object**.

They are defined **inside a class** and usually use the `self` keyword to access object or class data.

In simple words:

Functions outside a class → functions

Functions inside a class → methods

- Methods are used to **perform actions on object data**
- `self` represents the **current object**
- Methods can:
 - Access **instance attributes**
 - Access **class attributes**
 - Return values or just perform an action (print, update, etc.)

```
class Car:
    name = "BMW" # Class attribute

    def __init__(self, model, year):
        self.model = model # Instance attribute
        self.year = year

    def welcome(self):
        print("Welcome to BMW")
```

```
def get_model(self):  
    return self.model
```

```
def get_name(self):  
    return self.name
```

```
c1 = Car("X5", 2020)
```

```
c1.welcome()  
print(c1.get_name())  
print(c1.model)  
print(c1.get_model())  
print(c1.year)
```

Welcome to BMW

BMW

X5

X5

2020

Ex →

```
# Calculate marks of 3 subjects
```

```
class Student2:
```

```
    def __init__(self,name,sub1,sub2,sub3):
```

```
        self.name = name
```

```
        self.sub1 = sub1
```

```
        self.sub2 = sub2
```

```
        self.sub3 = sub3
```

```
    def get_avg(self):
```

```
        print("Average is :", (self.sub1+self.sub2+self.sub3)/3)
```

```
stu1 = Student2("Raj",85,90,95)
```

```
print(stu1.name)
```

```
stu1.get_avg()
```

Raj
Average marks: 90.0

```
class Student2:
    def __init__(self,name,marks):
        self.name = name
        self.marks = marks
    def get_avg(self):
        sum = 0
        for mark in self.marks:
            sum += mark
        print("Average marks:", sum/len(self.marks))

stu1 = Student2("Raj",[85,90,95])
print(stu1.name)
stu1.get_avg()
```

Raj
Average marks: 90.0

Static Methods →

static method is a method that **does not depend on object data**
It works at the **class level**, not on individual objects.

- Static methods **do not use** `self`
- They **cannot access instance attributes directly**
- Used for **utility or helper logic** related to the class
- Defined using the `@staticmethod` decorator

Why use Static Methods?

- When logic **does not need object data**
- For:
 - Validation methods
 - Utility/helper functions

- Common messages or calculations

```
class Student2:

    @staticmethod
    def welcome():
        print("Welcome to Student class")

    def __init__(self,name,marks):
        self.name = name
        self.marks = marks
    def get_avg(self):
        sum = 0
        for mark in self.marks:
            sum += mark
        print("Average marks:", sum/len(self.marks))

stu1 = Student2("Raj",[85,90,95])
Student2.welcome()
print(stu1.name)
stu1.get_avg()

Welcome to Student class
Raj
Average marks: 90.0
```

Abstraction →

Abstraction means **hiding unnecessary implementation details** and showing **only what the user needs to know**.

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass
```

```

class Dog(Animal):
    def sound(self):
        print("Bark")

class Cat(Animal):
    def sound(self):
        print("Meow")

d = Dog()
d.sound() # Bark

```

Why this is abstraction?

- `Animal` defines **what must be done** → `sound()`
- `Dog` and `Cat` decide **how it is done**
- User just calls `sound()` without caring about the logic inside

Implementation is hidden

Only required behavior is exposed

Code becomes easier to extend and maintain

Encapsulation →

Encapsulation means **binding data and methods together into a single unit (object)** and controlling how the data is accessed.

Ex →

Account question

```

class Account:
    def __init__(self, acc_no, balance):
        self.acc_no = acc_no
        self.balance = balance
        print("welcome to your account - ", self.acc_no)
        print("Your current balance is - ", self.balance)

    def deposit(self, amount):

```

```

        self.balance += amount
        print("Amount deposited:", amount, "\nNew balance:", self.balance)

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance")
        else:
            self.balance -= amount
            print("Amount withdrawn:", amount, "\nNew balance:", self.balance)

acc1 = Account("123456", 10000)

acc1.deposit(7000)
acc1.withdraw(5000)

welcome to your account - 123456
Your current balance is - 10000
Amount deposited: 7000
New balance: 17000
Amount withdrawn: 5000
New balance: 12000

```

Why is this Encapsulation?

- Account data (`balance`) is **not changed directly**
- All updates happen through **methods**
- Logic and data stay **together inside the object**

Data is protected

Business rules are enforced

Code is safer and cleaner

del keyword →

The `del` keyword is used to **delete object attributes or the object itself**.

del s1,name → Removes the `name` attribute from the object `s1`.

del s1 → Completely deletes the object from memory.

After deleting:

- Accessing the attribute or object will raise an error
- Memory used by the object can be freed

Private (like) attributes and methods →

Python does not have **true private variables**, but it supports **name mangling** using `__` (double underscore).

Attributes or methods starting with `__`:

- Are **meant to be used only inside the class**
- Cannot be accessed directly from outside the class

```
class Person:
    __name = "Not Set"
    __age = 0

    def __init__(self, name, age):
        self.__name = name
        self.__age = age

    def __welcome(self):
        print("Welcome to the Person class", self.__name)

    def introduce(self):
        self.__welcome()

p1 = Person("Raj", 20)
Person.introduce(p1)      # Will give O/P
print(Person.__age)       # Give error as we are calling Private attr from o
                           # outside
p1.__welcome()            # Give error
```

This prevents **accidental access or modification** from outside the class.

Used for data protection

Improves code safety

Encourages proper usage through methods

Inheritance →

Inheritance allows a child class to **reuse properties and methods** of a parent class.

```
class Car:
    .....
    .....

class Tata(Car):
    .....
    .....
```

Types →

Single level Inheritance →

One parent → one child

```
class Car:
    pass

class Tata(Car):
    pass
```

Multi Level Inheritance →

Grandparent → Parent → Child

```
class Vehicle:
    pass

class Car(Vehicle):
    pass

class Tata(Car):
    pass
```

Multiple Inheritance →

One child inherits from **multiple parent classes**

class C(A, B ,C):

```
class A:
    pass

class B:
    pass

class C(A, B):
    pass
```

Super Method →

`super()` is used to **access methods or constructors of the parent class** from a child class.

It is mainly used when:

- You want to **extend parent functionality**
- You want to call the **parent constructor (`__init__`)**

```
class Car:
    def __init__(self, brand):
        self.brand = brand

    def show_brand(self):
        print("Brand:", self.brand)

class Tata(Car):
    def __init__(self, brand, model):
        super().__init__(brand) # Calls parent constructor
        self.model = model

    def show_model(self):
        print("Model:", self.model)
```

```
c1 = Tata("Tata", "Nexon")
c1.show_brand()
c1.show_model()
Output
```

```
Brand: Tata
Model: Nexon
```

Class Method (`@classmethod`) →

A **class method** works with the **class itself**, not individual objects.

- Uses `cls` instead of `self`
- Can access and modify **class variables**

```
class Company:
    company_name = "OpenAI"

    @classmethod
    def change_name(cls, name):
        cls.company_name = name

Company.change_name("TechCorp")
print(Company.company_name)
Output
```

```
TechCorp
```

Property Decorator

The `@property` decorator lets you **access a method like an attribute**.

```
class Person:
    def __init__(self, age):
```

```

        self._age = age

    @property
    def age(self):
        return self._age

p = Person(25)
print(p.age

```

Polymorphism →

Polymorphism means **one thing behaving in different ways**.

Same function or operator works differently based on context.

```

print(7 + 8)           # Addition
print("Hello" + " World") # String concatenation
print([1,2,3] + [4,5,6]) # List merge

```

Even though `+` is the same operator, its **behavior changes**.

Dunder (Magic) Methods

Python internally uses **special methods** to perform operations.

Operation	Method
<code>a + b</code>	<code>a.__add__(b)</code>
<code>a - b</code>	<code>a.__sub__(b)</code>
<code>a * b</code>	<code>a.__mul__(b)</code>

```

class Vector:
    def __init__(self, x):
        self.x = x

    def __add__(self, other):

```

```
return self.x + other.x
```

```
v1 = Vector(10)
```

```
v2 = Vector(20)
```

```
print(v1 + v2)
```

```
30
```